

Reviewed the uncommitted duplicate-line-item changes (SQL, C# services, Proposal + Revised Estimates JS) and ran a small dedupe simulation. Findings below, highest severity first.

Page Name	Bug Description	Severity	Suggested Fix
Revised Estimates / Budget-to-Actual	<code>spEnsureUniqueProposalLines</code> deletes duplicate <code>ProposalLines</code> by parent + name, but does not reassign <code>EstimateMappings</code> (or schedule mappings) from the deleted line's <code>EstimateCategoryId</code> to the kept line. Costs can vanish or attach to an orphaned category. Runs on every ETA load.	High	Before deleting losers: move mappings to the surviving <code>EstimateCategoryId</code> , then delete. Or delete only unmapped duplicates.
Revised Estimates / Budget-to-Actual	After the new parent+name uniqueness rule, <code>spCleanUpDuplicateProposalLinesAndEstimateCategories</code> still partitions duplicates by Name only . Same item name under different parents (e.g. Materials > Labor and Framing > Labor) can be treated as duplicates; the unmapped one is deleted incorrectly.	High	Update cleanup to partition by <code>ParentEstimateCategoryId</code> + trimmed name (same key as the new uniqueness rule).
Proposal	Frontend <code>#deduplicateLineItemsByName</code> keeps the higher amount and drops the other; totals can shrink with no warning (e.g. 100 + 250 → 250). Backend <code>DeduplicateLineItemsByName</code> does the same on save/load.	Medium	Warn before collapsing, or sum amounts when merging; at least show a toast when rows are removed.
Revised Estimates / Budget-to-Actual	Client <code>#deduplicateLineItemsByName</code> sums <code>costToDate</code> but does not recalc <code>balance</code> / <code>percentage</code> . Simulation: Drywall 1000/200 + drywall 500/300 → <code>costToDate</code> 500 but <code>balance</code> stays 800 (should be 500) and % stays 20 (should be 50).	Medium	After merge, set <code>balance = revised - costToDate</code> and recompute <code>percentage</code> (match <code>EstimateService</code>).
Proposal	Backend <code>DeduplicateLineItemsByName</code> drops whitespace-only names; UI still keeps blank rows by <code>id</code> . Save can remove placeholder/in-progress rows without a clear error (required-name validation only checks empty string on visible inputs).	Medium	Keep blank names out of silent drop, or block save with an explicit "Item Name is required" and don't strip them server-side without feedback.
Proposal (API after save)	<code>EnsureUniqueProposalLinesAsync</code> runs raw SQL <code>DELETE</code> after <code>SaveChanges</code> , then <code>GetProposalAsync</code> loads via a tracked context that still holds deleted <code>ProposalLines</code> . Response can show stale/ghost lines until the scope ends.	Medium	Call <code>ChangeTracker.Clear()</code> (or detach/ <code>AsNoTracking</code> reload) after the proc, before <code>GetProposalAsync</code> .
Revised Estimates (map account)	<code>pl.Name.Trim()</code> / <code>selectedCategory.Name.Trim()</code> throw if <code>Name</code> is null → mapping save can 500.	Medium	Use null-safe compare: <code>(pl.Name ?? "").Trim().ToLower()</code> (same pattern already used elsewhere).
Revised Estimates (map account)	If a same-name line already exists under the parent, code skips creating a proposal line and never applies <code>mapping.Amount</code> to the existing line. Mapping is saved; amount from the new category flow is ignored.	Low	When reusing <code>existingLineWithSameName</code> , update its amount (or document that amount is ignored and clear it in the UI).

Test note: No automated unit tests covered this change set. A Node simulation confirmed proposal total loss (475 → 375) and stale ETA balance/percentage after client merge, plus the cleanup cross-parent delete risk.

I can patch these in priority order (High first) if you want.